

DAI: uma LLM de domínio específico com autoridade excluída do modelo, verificação por capacidades disjuntas e recibos recomputáveis

Hiago Kin Levi
Decripte · hiago@decripte.us

Este paper descreve a DAI (Decripte Artificial Intelligence), uma LLM própria de domínio específico inserida em uma arquitetura defensiva governada. Explica o modelo e seu ciclo de treinamento junto à extração estruturada de alvos, contratos de ação declarados, admissibilidade determinística, execução delimitada, observação independente, recibos selados e revisão e reconciliação posteriores.

Edição	Preprint após o depósito, preparado em 1 de outubro de 2026
Manuscrito	Corpo técnico preservado do manuscrito fornecido
Pedido de patente	Pedido provisório nos EUA 64/166,855; patente pendente
Status científico	Preprint; sem alegação de revisão por pares
Idioma	Português; tradução do manuscrito em inglês

Os experimentos e as observações de implantação relatados correspondem à versão e ao período descritos no manuscrito. Os resultados formais dependem das hipóteses declaradas. Esta edição não afirma que toda construção teórica esteja implementada ou validada de forma independente, nem que o software esteja disponível publicamente.

© 2026 Hiago Kin Levi. Todos os direitos reservados. A disponibilidade pública deste manuscrito não concede licença sobre código proprietário, pesos do modelo ou direitos de patente. Não se declara endosso científico de terceiros.

DAI: uma LLM de domínio específico com autoridade excluída do modelo, verificação por capacidades disjuntas e recibos recomputáveis

Hiago Kin Levi

Tradução em português do manuscrito correspondente em inglês. O corpo técnico foi preservado; esta edição atualiza a identificação e o status do pedido de patente.

Edição após o depósito. Pedido provisório nos EUA 64/166,855, recebido em 1 de outubro de 2026. Patente pendente; ainda não concedida. DAI é a LLM própria e sua arquitetura defensiva governada; ADAI nomeia a categoria técnica proposta.

Resumo

Descrevemos a DAI (Decripte Artificial Intelligence), sistema em produção no qual um agente de software conduz um episódio de resposta a incidente de ponta a ponta — curando um sinal, extraindo um alvo da evidência estruturada, escolhendo um playbook revisado, emitindo um contrato de ação, tendo esse contrato julgado por uma função de decisão pura nas suas entradas declaradas, executando numa borda de rede, verificando por uma credencial demonstravelmente incapaz de escrever, selando uma transcrição encadeada por hash, e voltando periodicamente para revisar, reconciliar e desfazer — sem humano aprovando passo algum.

As contribuições do sistema como artefato são: (i) uma função de decisão $\delta(a, \lambda, h)$ cujo domínio exclui toda saída aprendida, tornando a independência do modelo consequência da assinatura e não de checagem em tempo de execução; (ii) segregação de funções realizada como **disjunção de capacidades** entre duas credenciais, conferida empiricamente contra a API do provedor de aplicação antes da implantação — a credencial verificadora recebe erro de autenticação ao tentar escrever; (iii) um recibo que satisfaz um *contrato de recomputação*, recomputado em produção por um verificador que não importa código algum do sistema, com localização exata de adulteração demonstrada por alteração controlada; (iv) **perímetros por principal dentro de um domínio de aplicação compartilhado**, exigindo predicados de contenção que conjugam um discriminador de hostname com o endereço de origem, e um esquema de agregação que multiplica a capacidade de contenções simultâneas pelo limite de tamanho da expressão, sob um orçamento de regras da plataforma; (v) uma **reconciliação identificada por elemento** que permanece correta sob essa agregação, enquanto o algoritmo convencional identificado por regra é incorreto nas duas direções; e (vi) um modelo de linguagem de domínio treinado exclusivamente sobre as transcrições de desfecho verificado do próprio sistema, que narra e é estruturalmente incapaz de decidir.

Relatamos medições de produção: uma contenção executada, verificada por credencial distinta, e selada 887 ms após o primeiro turno do episódio, incluindo duas idas e voltas ao provedor de aplicação; um recibo de doze turnos recomputado externamente com todos os elos válidos e, sob alteração de um único

caractere no turno zero, falhando exatamente no índice alterado; isolamento com raio de alcance unitário confirmado pela ausência de qualquer regra de domínio para o endereço contido; e o ciclo completo testemunho → revisão → desfazer fechando. Relatamos também dez defeitos encontrados no período, todos silenciosos, e analisamos o que cada um revelou.

Palavras-chave: defesa cibernética autônoma, sistemas agênticos, isolamento entre tenants, verificação por capacidade disjunta, recibos à prova de adulteração, reconciliação, modelos de linguagem de domínio.

1. Introdução

1.1 Escopo

A DAI automatiza as fases de detecção, contenção e recuperação do ciclo padrão de tratamento de incidentes [3]. Ela defende infraestrutura do próprio operador e de dois produtos irmãos da mesma organização. Ingere sinais de analytics de borda de rede e de um serviço de detecção de ameaças em nuvem, abre um episódio por sinal material e — quando a evidência estruturada oferece alvo acionável e um playbook revisado cobre o tipo do sinal — altera a borda de rede para conter a origem.

É **defensiva por construção**: o catálogo de ações expressáveis contém apenas operações de contenção e de desafio sobre perímetros que o operador controla. Não há reconhecimento, não há varredura, não há ação dirigida para fora. Isso é imposto por um portão de construção que roda antes da suíte de testes, pelo raciocínio de que, se a garantia de que o sistema não ataca foi afrouxada, nada do que a suíte relatar depois importa.

1.2 A DAI é uma ADAI, e isso é conferível

O artigo companheiro define **ADAI** — *inteligência artificial de defesa autônoma* — não como capacidade e sim como classe com quatro critérios de pertencimento. A DAI é uma instância dessa classe. A afirmação não é retórica: cada critério é conferível, e três dos quatro o são sem a nossa cooperação.

Critério	O que exige	Como a DAI satisfaz	Onde se verifica
A Age em produção	Mudança de configuração em sistema com tráfego real	Cria e remove regras numa borda de rede que serve três serviços vivos	§9, primeira linha
B Decide sem aprovação concomitante	Para alguma classe de ação, a determinação é do sistema	Roda no nível de autonomia A4; zero episódios aguardando humano	§3.4, §9
C Admissibilidade julgada fora de quem propõe	O domínio da função de decisão exclui saída aprendida	$\delta(a, \lambda, h)$ — contrato, constante de implantação, um booleano; nenhum modelo na assinatura	§3.3
D Evidência que um não-operador verifica	Integridade decidível a partir do registro apenas	Recibo de doze turnos recomputado por verificador que não importa código nosso; turno alterado localizado exatamente	§4.4

O critério **D** é o que pediríamos a um leitor que testasse primeiro, por ser o mais barato de falsificar e o mais frequentemente afirmado sem base. Pegue um recibo, implemente um verificador a partir da descrição publicada nele, rode, depois altere um caractere e confirme tanto que o verificador reprova quanto que o índice reportado é o índice alterado. Relatamos o resultado de fazer exatamente isso na §4.4, inclusive que o **primeiro** recibo submetido ao teste falhou (§8.2).

Os mecanismos que este artigo reivindica — perímetros por principal num domínio de aplicação compartilhado (§5), verificação por capacidade disjunta (§5.4), o contrato de recomputação (§4.3), e as duas consequências da agregação (§5.7, §6.2) — estão expostos nessas seções, cada um com a medição ou a demonstração que o sustenta.

1.3 Quais problemas se revelaram difíceis

Não os previstos. Decidir *o que fazer* é comparativamente fácil: o espaço de respostas sensatas a uma origem hostil numa borda de rede é pequeno, e um catálogo revisado o cobre. Os problemas que sustentaram o peso foram:

1. provar, a quem não confia no sistema, que aquilo que ele diz ter feito é o que aconteceu (§4);
2. tornar estruturalmente impossível que o incidente de um principal altere o perímetro de outro, numa implantação em que os três dividem um domínio de aplicação (§5);
3. garantir que nenhum efeito criado no mundo fique órfão do seu registro, inclusive quando o processo morre entre agir e persistir (§6);
4. manter um componente aprendido no laço sem deixá-lo perto da decisão (§7).

1.4 Notação

Reutilizamos a notação do artigo conceitual companheiro: P principais, $\Pi : P \rightarrow 2^{\mathcal{W}}$ o mapa de perímetro, $\mathcal{G}$ os predicados de regra expressáveis no ponto de aplicação, $a = \langle \theta, x, \pi, \sigma, \rho, \ell \rangle$ uma ação, $B(a)$ o raio de alcance, $\mathcal{E}$ o envelope de ação, δ a função de decisão, κ a serialização canônica, H o hash, d_i os digestos de turno, D o selo.

2. Arquitetura

2.1 Camadas

A decomposição segue a separação entre ponto de decisão e ponto de aplicação de política, do controle de acesso [22, 25], e o estagiamento Monitorar–Analisar–Planejar–Executar da computação autônoma [15, 13], com as duas adições que o arcabouço companheiro exige: um portão entre Planejar e Executar cujo domínio exclui saída aprendida, e um caminho de retorno mundo $\rightarrow$ registros.

Camada	Responsabilidade
domínio	tipos de valor: nível de autonomia, escopo de efeito, reversibilidade
tempo real	contratos de papel e autoridade; esquema de eventos do episódio
orquestração	curadoria, extração de evidência, planejamento, política, transcrição, condutor, revisão, fila, mapa de perímetros
conectores	SDK de conectores; adaptadores de borda; adaptadores de ingestão; reconciliação
api	superfície HTTP, repositório, recibo, testemunho, entrada de sinal
llm	tokenizador, arquitetura, corpus, treino, narrador, inferência
laboratório	achados sobre o corpus; mede, nunca muda
chaves de api	credenciais de consumidores de sistema
pontos de entrada	processos

Duas regras estruturais são impostas em tempo de construção. A propriedade de camada é declarada e conferida, de modo que a árvore não pode crescer para dentro de camada que pertence a um sistema irmão. E o SDK de conectores, duplicado entre duas bases de código por exceção explícita e rastreada, tem o digesto da árvore sintática normalizada de cada módulo comparado, de modo que uma mudança de lógica de um lado só reprova a construção — a alternativa sendo que uma correção num conector alcance um de dois caminhos de código e nada sinalize a divergência.

2.2 Processos

Processo	Papel
API	HTTP: console, entrada de sinal, recibos, testemunho. Alcançável por túnel; sem porta pública no host.
Núcleo	Consome a fila de episódios e os conduz. Detém as credenciais de borda.
Observador de borda	Agendado: analytics de borda → sinais.
Observador de ameaças	Agendado: detecção de ameaças em nuvem → sinais.
Revisor	Agendado de hora em hora: revisa, desfaz, reconcilia, fecha episódios pendurados.
Narrador	Agendado: narra achados do laboratório.

Os processos API e Núcleo detêm papéis de execução **diferentes**. Compartilhavam um até que uma revisão encontrou que o processo exposto à internet podia consumir a fila de episódios e escrever no corpus, inclusive no prefixo de desfecho verificado usado como sinal de treino. É menor privilégio [24] aplicado na fronteira de processo: o processo exposto à rede recebe o menor conjunto de permissões que o deixa fazer o seu trabalho, e "é a mesma aplicação" não é razão para ampliá-lo.

2.3 Adversário assumido

Adotamos sem mudança o modelo de ameaças do artigo companheiro: o adversário é autor do conteúdo do sinal, escolhe tempo e volume, pode deter uma conta de operador comprometida, e observa as saídas do sistema; o adversário não detém a credencial executora, não consegue escrever abaixo da aplicação

no repositório de evidência nem no selo, não consegue calcular segundas pré-imagens, e não controla o provedor de aplicação.

A última exclusão é a hipótese não modelada mais significativa do sistema, e vale reenunciá-la em termos concretos: a verificação relê a borda pelo mesmo provedor que executou a mudança, usando uma segunda credencial desse provedor. Um provedor que respondesse às duas credenciais de forma consistente e falsa produziria status **verified** para uma contenção que não existe, e nada na arquitetura o detectaria. O que a construção de fato exclui é a falha muito mais frequente — *o sistema* estar errado sobre o que fez —, e a §8 mostra que ela não é hipotética.

2.4 Percurso do episódio

$$\text{sinal} \rightarrow \text{curadoria} \rightarrow \text{hipótese} \rightarrow \underbrace{\text{ext}}_{\text{evidência}} \rightarrow \underbrace{\text{cand}(\theta)}_{\text{playbook}} \rightarrow a \rightarrow \delta \rightarrow \underbrace{\text{exec}(c_e)}_{\text{executor}} \rightarrow \underbrace{\text{verify}(c_v)}_{\text{verificador}} \rightarrow \text{selo}$$

com um caminho de retorno agendado

$$\text{selo} \rightarrow \text{revisão} \quad \text{e} \quad \text{mundo} \rightarrow \text{reconciliação} \rightarrow \text{registros}.$$

Cada seta escreve um turno. A transcrição é o artefato primário: um efeito sem transcrição é tratado como defeito a reconciliar (§6), e não como ação bem-sucedida com registro incompleto.

2.5 Os doze mecanismos, e onde cada um está

O artigo companheiro nomeia doze mecanismos e liga cada um à definição ou ao teorema que lhe dá sentido. Esta tabela é a outra metade: **onde cada um vive neste sistema, e o que se mede sobre ele.**

Ela existe porque um artigo de arcabouço nomeando doze mecanismos e uma implementação carregando onze é uma lacuna que ninguém vê — o décimo segundo continua sendo discutido em prosa, e prosa se lê como descrição.

Sigla	Onde está	Medido
AC	Função de decisão determinística sobre o contrato de ação	12 violações construídas adversarialmente, todas recusadas (EIR = 1)
MEA	A assinatura de δ : contrato, constante de implantação, um booleano	Um portão de construção recusa um quarto parâmetro, e recusa SDK de modelo fora de uma camada confinada
ACO	O objeto de contrato, validado antes de chegar ao ponto de decisão	Violação construída: autonomia máxima com rollback vazio — recusada
EBT	Extração de alvo sobre conjunto fixo de chaves; texto livre fora do domínio	Violação construída: endereço nomeado só em campo influenciado pelo adversário — recusada. E ver §8.10
PPB	Perímetro resolvido do principal da sessão por igualdade estrita	17 testes sobre o mapa de perímetros; não há caminho padrão a construir
DSC	Predicado de contenção conjugando hostname com endereço de origem	Zero regras de domínio criadas para o endereço contido
SAV	Duas credenciais no provedor de aplicação, de capacidade disjunta	A credencial verificadora recebe erro de autenticação ao tentar escrever
SSC	Digesto sobre dez campos incluindo o do antecessor; selo guardado à parte	12 elos recomputados válidos; 36 alterações construídas, todas detectadas
PPR	Recibo exportado carregando o contrato de recomputação inteiro	Taxa de recomputação 1; localização exata nas 23 em que um conjunto falhou
ELR	Reconciliação chaveada pelo elemento contido	Relata três contagens — reconciliadas, órfãs, inconferíveis — nunca duas
RBT	Rota de testemunho gravando evidência; veredito determinístico a lê	Ciclo testemunho→revisão→desfazer completo, regra removida e confirmada ausente
SRL	Revisão agendada abrindo sessão selada nova que cita a original	Roda de hora em hora; a original nunca é reaberta

Para que serve esta tabela, e a linha que não é confortável

Um leitor pode pegar qualquer linha e pedir o artefato. É esse o ponto: o inventário é finito, então a pergunta *quais dos doze este sistema implementa* tem resposta, e a resposta é **errada quando é errada**.

EBT é a linha desconfortável. A garantia vale por construção — o domínio da função de extração exclui texto livre, e o arnês o exercita. O que **não** vale é a verificação por terceiro: a transcrição registra o resumo do sinal, não a evidência estruturada que o parser leu. Quem recebe o recibo confirma contrato, decisão, efeito e verificação, e precisa **acreditar em nós** quanto à origem do alvo.

É exatamente o tipo de confiança que o recibo existe para dispensar, e a §8.10 o relata como o defeito aberto que é, não como limitação da medição.

3. O envelope, implementado

3.1 Contrato e catálogo

Uma ação é proposta como contrato com campos declarados: fase, tipo de ação, tipo de alvo, referência do alvo, parâmetros, pré-condições, pós-condições, passos de rollback, nível de risco, reversibilidade, nível de autonomia exigido, escopo de efeito, chave de idempotência, tempo limite.

A construção é por instanciação a partir de um catálogo $\mathcal{C}$ de playbooks revisados — o modelo de orquestração [14] preservado deliberadamente —, com $\text{cand} : \Theta \rightarrow 2^{\mathcal{C}}$ mapeando tipos de sinal para conjuntos de candidatos. Um papel de autoridade superior pode expressar preferência *dentro* de $\text{cand}(\theta)$; não pode introduzir candidato. A imagem da construção é portanto $\bigcup_{\theta} \{\text{instantiate}(c, x) : c \in \text{cand}(\theta)\}$ independentemente de qualquer saída aprendida.

O catálogo atual admite três tipos de sinal e ações de contenção sobre alvos de endereço apenas. Um tipo de sinal fora de $\text{dom}(\text{cand})$ produz um episódio que investiga e registra a lacuna: ausência de playbook é desfecho legítimo, não erro a contornar, e a lacuna registrada é a entrada do laboratório para propor um playbook novo a um humano.

3.2 Alvos ancorados

A extração lê o campo de evidência estruturada do sinal, parseado como JSON, sobre um conjunto fixo de chaves. Campos de texto livre são ignorados deliberadamente: agir sobre um identificador que aparece em prosa influenciada pelo adversário é a ameaça de injeção indireta de prompt [10, 23, 7] expressa como efeito e não como saída.

Formalmente, os tipos acionáveis são $K_A = \{\text{ip}\}$, enquanto os tipos reconhecidos $K \supsetneq K_A$ incluem número de sistema autônomo e país. Estes últimos são registrados como *observados* e nunca oferecidos, porque o raio de alcance deles é a base inteira de clientes de um provedor — frequentemente incluindo o principal que está sendo defendido. Nenhum predicado do envelope sobre um único principal consegue limitar isso, de modo que a resposta correta é que aquele tipo de alvo nunca vire alvo.

3.3 A função de decisão

A decisão é

$$\delta(a, \lambda, h) \in \{\text{PERMITIR}, \text{NEGAR}, \text{APROVAR}\},$$

função pura do contrato a , do teto de implantação λ , e de um booleano h que registra a presença de um turno de autorização emitido por papel com autoridade de autorizar. h é **entrada, não inferência**: o ponto de decisão não deriva aprovação do conteúdo do contrato. O módulo não chama modelo algum; as regras são avaliadas da mais restritiva para a menos, e a primeira que casa decide.

Duas regras merecem enunciado porque são recusas que o sistema faz contra si mesmo:

$$\ell(a) = A_4 \wedge \rho = \varepsilon \implies \text{NEGAR}$$

— o tipo do contrato admite rollback vazio; a política não. Autonomia máxima sem caminho de volta é inadmissível por mais estreito que seja o escopo.

$$\ell(a) \succ \lambda \implies \text{NEGAR com motivo registrado}$$

— e o motivo é registrado como turno, de modo que uma recusa é tão auditável quanto uma ação.

Como o domínio de δ exclui saída de modelo, a independência do modelo é propriedade da assinatura. Nenhuma checagem em tempo de execução a impõe e nenhuma é necessária: um modelo que produzisse uma justificativa persuasiva para uma ação inadmissível não mudaria nada, porque a justificativa não é argumento que δ aceite.

3.4 Teto

Produção roda em A_4 . O teto é fixado no ambiente de implantação; o código traz um padrão. Os dois divergiram uma vez — um episódio narrou "autonomia A_3 acima do teto A_2 " depois de o padrão do código já ter mudado para A_4 —, então a implantação passou a reafirmar o valor do ambiente a cada publicação, em vez de depender de alguém lembrar.

4. Evidência

4.1 Construção da cadeia

A construção é o log encadeado por hash clássico [11, 27, 5, 18, 17], especializado de modo que o campo de encadeamento fique *dentro* do objeto hashado.

$k = 10$ campos são hashados, em ordem fixa: papel do ator, autoridade, instante de criação, natureza, chave do modelo, versão do modelo, carga, digesto anterior, sequência, identificador do episódio. A serialização canônica ordena as chaves, usa os separadores `,` e `:`, desabilita o escape de não-ASCII e coage valores não serializáveis por regra fixa.

$$\tau_i.\text{prev} = \begin{cases} \perp & i = 0 \\ d_{i-1} & i > 0 \end{cases}, \quad d_i = H(\kappa(\tau_i)), \quad H = \text{SHA-256}.$$

O digesto anterior é um dos dez campos hashados, de modo que o encadeamento é *interno* ao objeto serializado, e não por concatenação. Um verificador precisa, portanto, reconstruir o objeto; é por isso que a lista de campos exportada precisa ser completa (§4.3).

O registro do episódio guarda o selo D à parte, sob uma restrição de banco que recusa marcar um episódio como selado enquanto D estiver ausente. A restrição já disparou em produção: o sistema criou uma regra real na borda e em seguida falhou ao persistir o episódio, porque o caminho de inserção não preenchia o carimbo de selagem. A restrição converteu uma inconsistência silenciosa em falha ruidosa — e produziu o efeito órfão que motivou a §6.

Inserção é a única mutação admissível, imposta por gatilho de banco abaixo da aplicação. O gatilho já foi exercido a sério: recusou uma migração que teria reescrito o identificador de principal de 48 episódios históricos. A resposta correta foi corrigir para frente em vez de contorná-lo, e registramos que a tentação de contornar foi real — aqueles 48 episódios carregam um identificador que não corresponde a principal nenhum, e continuam carregando.

4.2 Caminho da verificação

A verificação não confia no que a escrita devolveu. Ela relê pela credencial verificadora e julga a pós-condição. Seja R_p a regra observada para o principal p , X_p o conjunto contido dela, e μ o modo contratado.

$$\text{status} = \begin{cases} \text{verificado} & R_p \neq \perp \wedge x \in X_p \wedge \text{mode}(R_p) = \mu \\ \text{inconclusivo} & R_p \neq \perp \wedge x \in X_p \wedge \text{mode}(R_p) \neq \mu \\ \text{falhou} & R_p \neq \perp \wedge x \notin X_p \\ \text{falhou} & R_p = \perp \end{cases}$$

As duas linhas de **falhou** são estados distintos com o mesmo veredito e diagnósticos diferentes: na terceira o perímetro está defendido e este alvo não está nele; na quarta o perímetro não carrega regra nossa nenhuma, o que também significa que a reconciliação da §6.2 não encontrará nada para adotar. Colapsá-las perderia a distinção exatamente onde um operador precisa dela.

Ausência é **falhou**, não **inconclusivo**: o que se prometeu foi que a contenção existisse e valesse. Severidade diferente da contratada é **inconclusivo**: houve contenção, mas não a contratada, e chamar isso de **verificado** ou de **falhou** seria enunciar errado. A assimetria é deliberada — **inconclusivo** fica reservado para *o mundo discordar sobre grau*, nunca sobre existência.

4.3 O recibo

O recibo exportado carrega: metadados do episódio; a cadeia completa com os dez campos hasheados por turno; turnos destacados (contrato, decisão, execução, verificação independente); **tanto** o selo guardado **quanto** o digesto recalculado da cauda da cadeia, separadamente, para que a divergência fique visível; o algoritmo — função, lista ordenada de campos, regras de serialização, regra de encadeamento; sete passos numerados de verificação; e um enunciado explícito dos limites.

O enunciado dos limites faz parte do artefato por desenho. O recibo estabelece *processo*: que estes turnos, nesta ordem, com este conteúdo, foram registrados e não alterados depois; que o contrato precedeu a execução; que a verificação foi registrada sob papel distinto do executor. Ele não estabelece *origem*: que o sinal era verdadeiro, ou que o mundo correspondia à descrição — a distinção entre integridade e veracidade de um registro que a literatura de proveniência torna explícita [21]. O recibo diz isso, para que quem o leia não extraia dele afirmação que ele não sustenta.

4.4 Recomputação externa, medida

Um verificador escrito a partir da descrição publicada no recibo, sem importar código algum do sistema, foi rodado contra um recibo de produção de doze turnos:

Cadeia	Validade por turno	Cauda = selo
intacta	[True × 12]	True
carga do turno 0 alterada em um caractere	[False, True, True, ...]	False

A segunda linha é o comportamento que distingue uma cadeia de uma soma de verificação: o turno 0 falha no próprio digesto e todo turno seguinte falha no elo com o digesto anterior, de modo que o menor índice que falha é exatamente o índice alterado. A localização de adulteração é exata.

Este teste é a razão de a §8.2 existir. O primeiro recibo submetido a ele **falhou**, em todos os turnos, e falhou por uma razão que nenhum teste interno poderia ter encontrado.

5. Perímetros por principal num domínio de aplicação compartilhado

5.1 O problema, concretamente

O isolamento multi-tenant é normalmente estudado nos planos de dados e de armazenamento [16]; aqui o recurso compartilhado é o próprio ponto de aplicação.

Três serviços — o console do operador, um produto de gestão, e um produto capaz de operar offline — são servidos a partir de uma zona DNS atrás de uma rede de borda. Os três resolvem para os mesmos endereços anycast, de modo que os três atravessam o mesmo ponto de aplicação.

O mecanismo disponível no início eram regras de acesso de domínio por origem: predicados da forma $\text{from}(x)$, independentes do destino. Pelo lema do discriminador, nenhuma regra dessas é tenant-sólida: uma regra levantada por causa do incidente de um produto nega serviço aos três, incluindo o console que um operador usaria para acompanhar o incidente acontecer.

5.2 Perímetro vinculado ao principal

O mapa de perímetro Π é declarado em código, não em configuração mutável, e resolvido pelo principal do episódio com **igualdade estrita e sem padrão**.

A escolha de código em vez de configuração é deliberada e é o mesmo argumento usado para os identificadores dos consumidores de sistema: este mapa decide *onde* uma credencial de contenção pode escrever. Num repositório mutável, quem tem escrita nele pode apontar o perímetro de um principal para o domínio de outro — e as próprias credenciais moram nesse repositório, de modo que o custo marginal da colocação mais estrita é zero enquanto o benefício marginal é que a mudança passa por revisão e aparece num diff.

Não existe "se o principal for desconhecido, use o perímetro do operador". Esse padrão é precisamente como o incidente de um principal vira indisponibilidade de outro.

5.3 Contenção escopada a hostname

O discriminador é o campo de host da requisição. A contenção é expressa como

$$g_p = (\text{host}(t) = h_p) \wedge (\text{src}(t) \in X_p), \quad X_p \subseteq \mathcal{X},$$

uma regra por principal carregando um conjunto de endereços, em vez de uma regra por contenção.

Capacidade. A plataforma admite L regras personalizadas, das quais k já estavam implantadas para proteções de produção não relacionadas, deixando orçamento $L - k$. Seja M o maior número de endereços que uma expressão comporta e $q = |P_Z|$ o número de principais.

$$\text{cap}_{\text{por contenção}} = L - k, \quad \text{cap}_{\text{por principal}} = \min(L - k, q) \cdot M, \quad \text{cap}_{\text{compartilhada}} = M.$$

A grandeza $N = \sum_p |\mathcal{X}_p|$ é a *demand* — contenções em vigor — e um arranjo é viável enquanto $N \leq \text{cap}$. Escrever a razão como $N/(L - k)$, como um rascunho anterior desta seção fazia, compara demanda com capacidade e não significa nada; a comparação correta é

$$\frac{\text{cap}_{\text{por principal}}}{\text{cap}_{\text{por contenção}}} = \frac{\min(L - k, q) \cdot M}{L - k} = M \quad \text{sempre que } q \geq L - k,$$

de modo que a agregação multiplica a capacidade pelo limite de tamanho da expressão, que na implantação da §9 é três ordens de grandeza. Note também que $\text{cap}_{\text{compartilhada}} = M$ é *menor* que $\text{cap}_{\text{por principal}}$ quando $q > 1$: o arranjo de orçamento mínimo não é o de capacidade máxima, porque todos os principais disputam uma expressão só. A regra compartilhada economiza vagas de regra e gasta vagas de endereço.

Agregação aqui não é otimização; sem ela o sistema não funciona.

Por que uma regra por principal, e não uma regra para todos. O arranjo de orçamento mínimo é uma *única* regra carregando o conjunto de cada principal como disjunção, $\bigvee_p[(\text{host}(t) = h_p) \wedge (\text{src}(t) \in X_p)]$, que custa uma vaga em vez de $|P_Z|$ e é igualmente tenant-sólida. Não o usamos, e a razão não é orçamento.

Nenhum dos dois arranjos alcança isolamento de credencial: o provedor escopa tokens de escrita ao **domínio**, de modo que em qualquer um deles o token executor de p é tecnicamente capaz de escrever qualquer regra da zona. O que o arranjo por principal compra é isolamento de falha, de dois tipos. Um defeito que corrompa a expressão escrita para p só consegue destruir X_p , ao passo que sob regra compartilhada ele destrói a contenção de todos de uma vez. E a escrita perdida da §5.7 passa a ser possível apenas entre dois episódios do *mesmo* principal, em vez de entre quaisquer dois episódios da instalação.

Gastamos, portanto, $|P_Z| - 1$ vagas de regra para dividir por $|P_Z|$ o raio de alcance de um erro de programação e o domínio de disputa. O artigo companheiro dá o enunciado geral; aqui vale notar que a escolha de orçamento mínimo é também aquela em que um único defeito desarma em silêncio a instalação inteira.

Ordenação dos endereços. X_p é serializado em ordem, de modo que o mesmo conjunto de contenção sempre produz o mesmo texto de expressão. Sem isso, a verificação relê uma cadeia de caracteres que difere só na ordem e relata divergência onde não há.

O adaptador nunca escreve o conjunto de regras inteiro. A API do provedor aceita a substituição do conjunto todo, e esse é o caminho de implementação mais curto; um defeito ali apagaria as k proteções preexistentes. Só existem operações por regra — criar, alterar, remover — no adaptador, e um teste reprova qualquer escrita fora dessa forma ou que toque regra que não carregue a marca do sistema.

Forma uniforme de observação. O adaptador de hostname devolve a mesma estrutura de observação que o adaptador de domínio — contagem de regras, modos, identificadores de regra. Deliberadamente: uma segunda forma exigiria que o condutor e o revisor soubessem qual adaptador produziu qual observação, e essa é exatamente a informação que o percurso do episódio não deve precisar ter. Os adaptadores são intercambiáveis na costura.

5.4 Capacidade de credencial, conferida empiricamente

Cada perímetro tem dois tokens. O executor escreve; o verificador lê. Ambos escopados a um domínio.

O menor privilégio é conferido contra a API do provedor antes de as credenciais entrarem em produção, em vez de ser inferido dos nomes de permissão. Para o par escopado a hostname, a tentativa do verificador de criar uma regra devolveu

```
{"success": false,
 "errors": [{"code": 10000, "message": "Authentication error"}]}
```

enquanto o executor criou e em seguida removeu com sucesso uma regra de sonda desabilitada, deixando intactas as k regras de produção preexistentes.

Isso é segregação de funções [4] realizada como disjunção de capacidades [8, 20] em vez de atribuição de papel [25], e é mais forte que uma garantia de caminho de código. "Quem verifica não executou", como propriedade da estrutura do programa, é quebrado por uma refatoração — e quebrado *em silêncio*: a verificação continua rodando, continua retornando sucesso, e agora não prova nada. Como propriedade da credencial, sobrevive à refatoração: um defeito que trocasse as duas instâncias de adaptador produziria erro de autenticação na escrita, e não verificação falsa.

5.5 Evidência de isolamento em produção

Um sinal entregue pela chave de API do produto offline, nomeando uma origem de varredura, produziu exatamente uma regra:

```
managed_challenge    {marca} · <hostname do produto>  
    (http.host eq "<hostname do produto>" and ip.src in {<endereço>})
```

com regras de acesso por origem no nível do domínio para aquele endereço: **0**. As k regras personalizadas preexistentes permaneceram intactas. O hostname do console não foi afetado *estruturalmente* — a expressão não consegue casá-lo —, e não incidentalmente.

O isolamento também foi demonstrado pela negativa, antes de o perímetro existir: um sinal do produto de gestão chegou ao executor e parou com "não há perímetro defendido registrado para este principal", e o alvo não foi bloqueado em lugar nenhum.

5.6 Uma correção que merece registro

Concluimos inicialmente, e relatamos, que o produto de gestão era indefensável naquele ponto de aplicação, com base em que um host de nome plausível carregava registro DNS não proxiado e resolvia para um único endereço de origem. As duas metades eram verdadeiras; a conclusão estava errada, porque aquele host não é o produto de gestão. O produto é servido no apex do domínio, que resolve para os endereços anycast da rede de borda.

O erro é metodologicamente interessante e o relatamos em vez de corrigi-lo em silêncio. Ele começou pela leitura de uma flag num host cujo nome foi *presumido* nomear o serviço, e o nome era plausível. Toda conferência subsequente confirmou fatos verdadeiros sobre o host errado — inclusive uma requisição HTTP que devolve cabeçalho de servidor da rede de borda para aquele nome por um caminho não relacionado. O que separa os casos é **resolver o nome e comparar com a faixa anycast**, e esse passou a ser o critério de registro. A tabela de resolução fica registrada ao lado do mapa de perímetros, para que o próximo leitor não precise repetir a medição.

A correção fortalece o argumento da §5.3: com o produto de gestão no *apex*, o escopo por hostname não é conveniência e sim necessidade, já que uma regra de domínio levantada pelo incidente dele cobriria o console e o produto offline juntos.

Ela também esgota o orçamento. Com L regras, k preexistentes e dois perímetros, $L - k - 2 = 0$. Um terceiro perímetro não falha em silêncio — o provedor recusa, o adaptador levanta, e o episódio narra que não conteve *depois de reler a borda* —, mas o sistema então detectaria e assistiria. Outro perímetro exige outro plano, não outro código.

5.7 A regra é estado mutável compartilhado, e escritor único é acidente de topologia

A agregação (§5.3) compra capacidade e custa atomicidade. Conter x para o principal p é leitura-modificação-escrita sobre um *conjunto*: ler X_p , calcular $X_p \cup \{x\}$, gravar. Os endpoints por regra do provedor não oferecem pré-condição de comparação-e-troca, de modo que dois episódios contendo endereços diferentes no mesmo host podem intercalar como

$$\text{ler}_1 \prec \text{ler}_2 \prec \text{escrever}_1 \prec \text{escrever}_2$$

e a segunda escrita, calculada antes de a primeira existir, descarta x_1 em silêncio.

A escrita não detecta. O provedor devolve sucesso; a regra foi atualizada exatamente como pedido. O que foi pedido é que foi calculado sobre estado velho.

A verificação independente detecta, sem conhecer o mecanismo: o episódio perdedor relê pela credencial verificadora, encontra o alvo ausente de X_p , e registra **falhou**. É a terceira linha da tabela da §4.2 chegando por uma causa para a qual ninguém a projetou, e é esse o argumento a favor de predicados estruturais em vez de checagens dirigidas.

Hoje a corrida é quase inalcançável, e esse é o problema. O núcleo roda uma tarefa e consome a fila em sequência, de modo que há um escritor só — mas isso é propriedade da *topologia de implantação*, uma contagem de processos e um laço, e não do adaptador. Deixa de valer no instante em que o serviço escala horizontalmente, e a tarefa horária de revisão, que também escreve no perímetro, já roda em processo separado. Depender de contagem de tarefas para não perder contenção é depender da coisa errada.

O adaptador passa então a conferir, depois de gravar, se o alvo de fato aparece no conjunto, e refaz a leitura-modificação-escrita **uma vez** se não aparecer. Uma tentativa, não um laço: se a segunda também perder, a verificação relê e reprova, que é o desfecho correto, ao passo que insistir mascararia contenção sustentada sobre um host — e contenção sustentada é sinal sobre a taxa de incidentes, não condição a absorver. Perder duas vezes não levanta exceção: levantar trocaria uma verificação reprovada e registrada por um erro, e o episódio perderia o turno de verificação, que é a prova.

Este defeito foi encontrado escrevendo o artigo companheiro, não lendo o código. Ele não tinha sintoma, e não teria nenhum até o primeiro dia em que dois episódios do mesmo principal se sobrepusessem.

6. Fechando o laço

6.1 Revisão

O revisor roda de hora em hora. Para cada contenção ele reúne três fatos observáveis — regra ainda presente (relida pela credencial verificadora), alvo reincidente desde a contenção, tempo decorrido — e aplica a função de veredito do artigo companheiro, com janela de não reincidência $W = 24$ h.

O veredito é registrado num episódio **novo** citando o identificador do episódio revisado, nunca anexado ao original. O original está selado, e reabrir um registro selado para escrever nele destrói a única propriedade que fazia o recibo dele valer alguma coisa.

CEDO_DEMAIS não abre episódio nenhum.

Desfazer passa pelo mesmo caminho de agir: o executor remove, o verificador confirma. Tratar rollback como "só limpeza" isentaria a classe mais consequente de ação das garantias que governam o resto.

Por principal. O revisor itera sobre todo perímetro registrado, usando o par de credenciais **daquele** principal. Revisar a contenção de um principal com o adaptador de outro releria o espaço de regras errado e relataria a contenção como ausente — produzindo **EXPIRADA** por engano e desfazendo contenção legítima. O par viaja, portanto, junto do principal, em vez de ser resolvido da configuração do processo.

6.2 Reconciliação

A revisão percorre registros $\rightarrow$ mundo, o que deixa a janela entre agir e persistir. Essa janela não é hipotética: ela se abriu em produção quando o sistema criou uma regra real e a escrita do episódio falhou logo em seguida, deixando uma regra na borda sem registro de execução, invisível para sempre à revisão.

A travessia mundo $\rightarrow$ registros é o problema de convergência da gestão declarativa de configuração [2, 12], sob uma restrição que aqueles sistemas não têm: o registro é probatório, de modo que a divergência não pode ser reparada reescrevendo o registro.

Perímetro de domínio (identificado por regra). Regras que carregam a marca do sistema são listadas pelo verificador e cada uma conferida contra os registros em três estados: reconciliada, sem episódio, sem registro de execução. O terceiro estado existe porque "existe um episódio?" mostrou-se grosseiro demais — a regra órfã *tinha* episódio, parado em estado de execução e sem recibo. O remédio difere por estado, então o nome também tem de diferir.

Perímetro de hostname (identificado por elemento). Aqui o mapa id de regras para episódios não existe: uma regra carrega $|X_p|$ endereços originados em $|X_p|$ episódios distintos, e a descrição é fixa por principal, não carregando informação por elemento. A unidade da reconciliação é, portanto, o endereço:

```
para p em principais_defendidos:
  X ← conjunto_contido(c_v(p))          # só a credencial verificadora
  se X =  $\perp$ : adiar; continuar           # ilegível  $\neq$  vazio
  para x em X:
    se  $\neg \exists$  episódio de p com (contrato mirando x  $\wedge$  recibo de execução)
       $\wedge \neg \exists$  adoção anterior de x por p:
        adotar(p, x, modo ← modo_observado(c_v(p), x))
```

Custo: $O\left(\sum_p |X_p|\right)$ testes de existência por ciclo.

Três detalhes sustentam o algoritmo, e cada um corresponde a um defeito observado: a leitura usa a credencial verificadora, porque reconciliar é observar; uma borda ilegível adia em vez de ser tratada como vazia, já que tratá-la como vazia conclui em silêncio "nada está contido" enquanto tratá-la como totalmente órfã adotaria o mundo; e o modo adotado é **lido da borda**, não assumido, porque registrar uma severidade padrão diferente da real faz a revisão seguinte comparar o que leu com o que registrou e concluir divergência onde não há.

Adoção, não reconstrução. Um efeito sem registro produz um episódio que registra o que é observado *agora*, cuja narração declara que é observação do estado atual e recusa explicitamente afirmar o instante de criação. Escrever um carimbo de criação plausível colocaria num registro probatório append-only uma

proposição que ninguém observou. O relógio de expiração começa na adoção, o que erra na direção de manter proteção em vez de removê-la cedo.

Sem adoção, o órfão é relatado a cada ciclo para sempre, sem nunca ser revisado nem expirar — ruído recorrente, que é como um relatório ensina o leitor a ignorá-lo.

6.3 Episódios pendurados

Um episódio em estado não terminal por mais de duas horas não avança sozinho: a mensagem que o abriu já saiu da fila, e o sinal viajava nela. Ele é abortado com o motivo escrito. Duas horas são deliberadamente generosas; um episódio de fato em curso dura segundos, e abortar um que ainda trabalha é pior que esperar. Isto também roda por principal.

Deixá-los abertos não é neutro: o console conta episódios parados, e uma população que nunca fecha é precisamente o ruído que ensina um operador a ignorar o console — a falha de fadiga de alarme documentada em operações de segurança [1] e em monitoração clínica [6].

6.4 Testemunho

Uma classificação registrada — falso positivo, confirmado, contexto — com a identidade do autor. Registra e nunca age. A revisão seguinte a lê e pode chegar a **FALSO_POSITIVO**, que remove a regra pelo caminho ordinário de ação.

O ciclo completo já rodou em produção: um humano registrou um falso positivo, a revisão agendada o leu, e a regra foi removida da borda e confirmada ausente pela credencial verificadora.

A distinção entre evidência e comando é propriedade de segurança. Uma interface em que a afirmação de uma pessoa atua diretamente sobre o perímetro é uma interface em que comprometer a conta dessa pessoa atua diretamente sobre o perímetro.

7. O modelo de linguagem de domínio

7.1 Restrição

Uma restrição de nível fundador exige que nenhum modelo de terceiro participe do caminho cognitivo — raciocínio, decisão, geração de contrato de ação, narração. A restrição não alcança síntese nem reconhecimento de voz, que continuam externos atrás de um gateway.

As motivações estão registradas e não são ideológicas. Pesos de terceiro carregam políticas de uso aceitável de terceiro, várias das quais excluem uso militar, o que é desqualificante para o cliente governamental do produto offline. E uma API externa não existe numa rede sem saída, que é onde aquele produto roda; inferência local precisa, portanto, ser propriedade do desenho e não adição posterior.

7.2 Construção

Tokenizador (codificação por pares de bytes [29] treinada no corpus do próprio sistema), Transformer causal [30], extração de corpus a partir de transcrições seladas, treino com divisão de validação **por episódio**, de modo que os turnos de um episódio nunca atravessem a divisão, narração com queda para

a regra determinística, e inferência.

Duas propriedades são estruturais.

Corpus de desfecho verificado. O treino lê apenas episódios com desfecho verificado. O sinal de aprendizado não é "o que o sistema disse", e sim "o que o sistema fez e o que uma credencial de capacidade disjunta confirmou". Qualidade de rótulo dominando tamanho de conjunto de dados é a lição recorrente do trabalho aplicado em sistemas de aprendizado de máquina [28]. Este é um conjunto muito menor e muito mais caro que a alternativa, e esse é o ponto: um corpus de afirmações não verificadas ensina um modelo a produzir afirmações.

Portão de aprovação. Um modelo treinado só é publicado depois de passar por um portão sobre tamanho e diversidade do corpus. Enquanto nenhum passa, o narrador cai para a regra determinística — e este é o estado normal, não estado de erro.

7.3 O que ele pode e o que não pode

Ele narra. Não pode decidir, pela §3.3: o domínio da função de decisão exclui saída de modelo. Um modelo que produzisse justificativa persuasiva para ação inadmissível não muda nada sobre admissibilidade.

A imposição em tempo de construção corresponde à afirmação. Um portão parseia a árvore sintática abstrata de cada módulo e recusa importações de pacotes de modelo de terceiro — recusando 21 raízes de pacote, inclusive carregadores de pesos pré-treinados, pelo raciocínio de que uma dependência de terceiro não precisa atravessar rede: pesos pré-treinados são a mesma dependência por outro caminho. Ele também recusa clientes de serviços de inferência do provedor de nuvem, que seriam inferência de terceiro sem instalar nada. Ele **não** recusa as bibliotecas de tensores e numérica: são as ferramentas com que o modelo próprio é construído, e recusá-las proibiria exatamente o que a restrição manda fazer.

O portão lê a árvore sintática e não o texto porque os próprios documentos deste projeto nomeiam os provedores que ele recusou usar; uma busca textual reprovaria a decisão de não os usar.

7.4 O que não podemos afirmar

Não podemos afirmar que o modelo bate a linha de base determinística num arnês adversarial, porque o arnês está projetado e não construído, e porque o corpus é pequeno. A restrição que aperta não é computação: episódios de desfecho verificado acumulam devagar por construção, e o plano da plataforma de aplicação não expõe os eventos de borda por endereço que mais os enriqueceriam. É o problema em aberto de escassez de corpus do artigo companheiro, encontrado na prática.

8. Defeitos encontrados, e o que cada um revelou

Todo defeito abaixo era **silencioso**: o sintoma era ausência de sinal, não presença de erro. Relatamo-los porque num sistema deste tipo são a parte mais informativa, e porque o argumento a favor da arquitetura é precisamente que ela converte falhas silenciosas em falhas ruidosas.

8.1 Perímetro não vinculado ao principal

O adaptador de borda era construído uma vez a partir da configuração do processo e usado em todo episódio, sem consultar o principal do episódio. O isolamento de leitura era completo e testado; nenhum principal jamais leu dado de outro. O caminho de ação não tinha checagem equivalente.

Latente enquanto um único principal produzia sinal; alcançável no momento em que chaves de API foram emitidas para dois produtos irmãos — que é para isso que as chaves existiam.

O que revelou. O isolamento tinha sido auditado inteiramente no caminho em que auditar é barato, porque toda consulta carrega um filtro visível, e assumido no caminho que altera o mundo. Auditar as consultas teria dado verde. A classe é geral: **garantia conferida onde conferir é fácil e assumida onde é caro.**

8.2 Um recibo que ninguém conseguia recomputar

O recibo reusava a projeção de exibição do console, que carrega cinco dos dez campos hasheados. A recomputação segundo as instruções do próprio recibo produziu cinco falhas em cinco turnos.

Todos os testes internos passavam, porque todos construíam a forma do turno à mão ou pela serialização do próprio sistema.

O que revelou. Uma propriedade de um artefato, testada através do produtor, é não testada. O remédio tem duas partes: o teste agora chama a projeção real em vez de reconstruir a forma dela, e a conformidade é demonstrada por um verificador que não importa código algum do sistema.

8.3 Uma consulta por um valor que o escritor nunca escreve

A consulta de revisão selecionava turnos cuja literal de natureza diferia por um prefixo daquela que o escritor persiste. SQL não reclama de literal que não casa com nada, então a junção lateral devolvia o conjunto vazio, em silêncio, por semanas.

Duas consequências. Contenções executadas **nunca eram revisadas** — o laço fechava apenas pelo ramo da adoção, que foi por que o ciclo de testemunho da §6.4 funcionou e deu falsa confiança. E a guarda contra readoção usava a mesma literal e era portanto permanentemente falsa, de modo que o mesmo efeito era readotado de hora em hora: os logs de produção mostram uma regra adotada às 20:44, 20:56, 21:56 e 22:55, com "reconciliadas: 0" em todas.

A mesma classe de erro existia em mais dois lugares: a "última contenção" da tela de conectores lia uma natureza que nunca existiu e era permanentemente nula, e o destaque de execução do recibo nunca era preenchido (a cadeia completa sempre esteve lá, de modo que a prova não foi afetada; o índice apontava para nada).

Nenhum teste pegou nada disso, porque todos usavam dublê de armazenamento, e um dublê devolve o que o teste manda — o risco clássico de testar com dublês [19]. O que estava errado era a *literal*, visível apenas contra o repositório real ou contra o texto do código.

Remédio. Os três agora derivam o valor de uma única enumeração, e a revisão o passa como parâmetro ligado. Um teste lê o código-fonte e reprova qualquer literal de natureza que o escritor não consiga produzir.

O que revelou. Um defeito na interface entre dois componentes, cada um individualmente correto e individualmente testado. Dublês codificam a crença do implementador sobre o outro lado; quando essa crença é o defeito, duplês são vendas.

8.4 Uma garantia documentada que nunca foi imposta

Um registro de decisão afirmava que um portão de construção cobria a camada do modelo e recusava SDKs de modelo de terceiro. O portão conferia outra coisa inteiramente. A ausência desses SDKs era de fato verdadeira — mas por disciplina, não por imposição.

Remédio. O portão descrito na §7.3 agora existe.

O que revelou. **Documentação que desliga a inspeção.** Quem lê "o portão cobre isso" para de conferir, e o custo aparece não quando a afirmação fica falsa, mas quando alguém confia nela. A regra de processo correspondente — toda garantia afirmada carrega uma checagem que a impõe, ou declara no próprio texto que é disciplina e não portão — passou a ser registrada como decisão em vez de ficar como hábito.

8.5 Perda silenciosa do endereço do adversário

Dois caminhos de acesso aos mesmos achados de ameaça de um provedor usam capitalizações de campo diferentes. O leitor usava uma; o caminho de entrega produzia a outra. Todo achado perdia o endereço de origem em silêncio — e o teste passava, porque o duplê dele misturava as duas grafias, forma que o provedor nunca emite.

Remédio. Fixtures capturadas literalmente da conta.

O que revelou. Um duplê que codifica a crença do implementador sobre um formato externo em vez do formato.

8.6 Componentes agendados rodando código velho

O pipeline de implantação atualizava dois serviços de longa duração e nenhuma tarefa agendada; cada definição de tarefa permanecia fixada no artefato que um humano registrou por último. A tarefa que fecha o laço de contenção e a tarefa que lê o feed de ameaças ficaram atrás da árvore testada — de modo que a correção da §8.5 foi mesclada, testada, implantada nos dois serviços e nunca chegou ao componente que a usava.

Remédio. Um único mecanismo de implantação cobre todo componente que roda, e relata o que mudou. Registrar uma revisão nova de tarefa basta, já que o agendamento aponta para a família e não para uma revisão fixada.

O que revelou. Divergência entre o artefato testado e o artefato em execução, por um caminho que nenhum teste observa.

8.7 Uma escrita perdida sem sintoma

A contenção sob agregação é leitura-modificação-escrita sobre um conjunto, sem comparação-e-troca disponível, de modo que dois episódios sobrepostos no mesmo principal podem perder uma contenção; a escrita relata sucesso, e só a verificação independente percebe. A §5.7 dá o mecanismo e a mitigação.

O que revelou. Duas coisas. Primeira: que uma decisão tomada por um motivo — agregar endereços numa regra para caber num orçamento — carrega consequências numa dimensão não relacionada, e que a segunda consequência é fácil de não ver justamente porque a primeira é a interessante. Segunda: que

"há um escritor só" era verdade e não era propriedade deste componente, e sim uma contagem de processos. Uma garantia que vale por causa de como algo está implantado é uma garantia que desaparece numa decisão de escala tomada por quem nunca leu este código.

Como foi encontrada. Escrevendo o tratamento formal da agregação para o artigo companheiro e notando que a mesma premissa produz um segundo teorema. Ela não tinha sintoma e não teria nenhum até a primeira sobreposição.

8.8 O recibo não autentica de quem é a sessão

O digesto cobre dez campos de cada turno. O principal não é um deles: o turno carrega o identificador da sessão, e o registro da sessão carrega o principal. Um terceiro recomputando um recibo conforme a §4.4 confirma, portanto, *o que aconteceu*, e não consegue confirmar *em nome de quem*.

O vínculo é real mas local — uma restrição append-only no repositório — e garantia local não vale nada para um leitor externo, que não tem acesso ao repositório nem como observar que a restrição existe. Num sistema cuja afirmação mais forte é isolamento por principal (§5), essa é a coisa errada a deixar fora do conjunto autenticado.

O que revelou. Que "o recibo prova X" e "o sistema impõe X" são frases diferentes, e que a segunda vinha sendo lida em silêncio como a primeira — por nós. A restrição append-only vinha sendo tratada como se tornasse a atribuição parte da evidência, e ela não torna; torna a atribuição parte da confiança do operador no banco dele.

Estado: não corrigido. Acrescentar o principal ao conjunto de campos hashados exige recalculando todo digesto histórico, que é exatamente o que um repositório append-only existe para impedir — é bifurcação da cadeia, não edição. Vincular o registro da sessão à carga do turno zero corrige para frente, deixa a história em paz, e é o que pretendemos fazer. Relatamos como não corrigido em vez de descrever a intenção como estado.

8.9 Validação de alvo mais estreita que o predicado que ela cumpria

A validação de alvo recusava endereços privados, de loopback, link-local, multicast, reservados ou não especificados — os endereços que não são origem roteável — e era tomada como cumprindo o predicado de escopo de efeito limitado. Ela admitia endereços dentro das **faixas publicadas do próprio provedor de aplicação**. Duas contenções contra endereços de proxy compartilhado do provedor ficaram em vigor na zona até serem encontradas por inspeção.

Nada no caminho de raciocínio errou, que é o que faz este defeito valer o espaço. A evidência era estruturada; a extração era sólida; o alvo proposto era o endereço que a evidência nomeou, e sob caminho proxiado a origem observada é o proxy. A ação estava, ainda assim, fora do predicado: um endereço de proxy compartilhado tem raio de explosão muito acima de um, já que o efeito recai sobre todo principal que chega por ele — inclusive o principal defendido — e é além disso o caminho pelo qual a contenção seria verificada, de modo que o efeito e a verificação dele passam a depender um do outro.

O que revelou. Que uma verificação pode existir, estar correta sobre o que verifica, e ser subconjunto próprio do predicado que lhe é creditado. A §8.4 era documentação afirmando uma verificação que não existia; esta é a imagem espelhada, e a mais difícil de enxergar: o subconjunto implementado é

exatamente o subconjunto que um revisor pensa em procurar. É também por que o portão de integridade de envelope ataca os predicados em vez de ler o código que os implementa — o defeito estava legível no fonte desde que o fonte existe.

Estado: corrigido. As faixas publicadas do provedor são enumeradas como dado estático e datado no caminho de decisão, com uma ferramenta que as atualiza por diff revisado. Estático e não resolvido no momento da decisão: uma consulta ao vivo faria δ depender de um serviço de rede e deixar de ser pura nas entradas declaradas, o que quebraria tanto a §3.3 quanto o contrato de recomputação. O caso está agora no arnês de envelope, onde a ausência dele era invisível.

As duas contenções em vigor são assunto à parte do defeito: removê-las é ação, e passa por revisão e desfazer como qualquer outra, e não por um operador editando a configuração do provedor por trás do sistema.

8.10 Uma garantia que vale por construção e não se confere pelo recibo

Evidence-Bound Targeting vale. O domínio da função de extração é um conjunto fixo de chaves sobre evidência estruturada; campos de texto livre não estão nele, e um ataque construído nomeando endereço apenas em campo influenciado pelo adversário é recusado.

O que não vale é a verificação por terceiro disso. O turno de curadoria registra o resumo do sinal, a severidade, o tipo e a referência de origem — não o `raw_content` que o parser leu. Conferido sobre um recibo exportado de produção: o endereço contido aparece no resumo da hipótese, no contrato de ação e na narração, e **em nenhum campo de evidência**, porque nenhum turno carrega um.

A consequência é exata. Quem recebe o recibo recomputa a cadeia, confirma que um contrato precedeu uma decisão que precedeu um efeito verificado por credencial disjunta — e então precisa **acreditar na palavra do emissor** sobre de onde veio o alvo. É a única coisa que o recibo existe para tornar desnecessária.

O que revelou. Que a garantia vale e a garantia é demonstrável a um estranho são afirmações diferentes, e um sistema pode satisfazer a primeira enquanto a segunda silenciosamente não se aplica. A distinção fica invisível até algo avaliar o recibo **contra os mecanismos**, e não só contra si mesmo: a cadeia fecha perfeitamente dos dois jeitos, porque o campo ausente nunca foi hashado — ele nunca esteve lá.

Estado: não corrigido. O reparo é registrar a evidência estruturada, ou um digesto dela, no turno de curadoria — trazendo-a para dentro do conjunto de campos hashados, de modo que a proveniência do alvo passe a ser recomputável. É mudança no que se hasheia, e portanto no contrato de recomputação, e é por isso que está relatada em vez de aplicada: o contrato é versionado, e recibo histórico precisa continuar conferindo sob a versão com que foi selado.

Até lá, EBT é relatado como *válido por construção e não verificável a partir do registro* — que é o que ele é, e é mais útil que qualquer uma das duas simplificações confortáveis.

9. Medições de produção

Implantação única, três serviços atrás de um domínio de aplicação compartilhado, ao longo de alguns dias. São instâncias, não estatísticas.

Observação	Resultado
Primeira contenção real	Regra criada na borda; verificada por credencial de capacidade disjunta; episódio selado
Duração do episódio, primeiro ao último turno	887 ms (12 turnos, duas idas e voltas ao provedor)
Recibo recomputado externamente (12 turnos)	todos os elos válidos; cauda = selo
Recibo com turno 0 alterado	falha no índice 0 e adiante; localização exata
Isolamento, perímetro de hostname	1 regra escopada a hostname; 0 regras de domínio para o mesmo endereço
Isolamento, caso negativo	sinal de principal não registrado: nenhuma ação, motivo registrado
Testemunho → revisão → desfazer	fechado; regra removida e confirmada ausente
Regra órfã	adotada; entrou no ciclo normal de revisão
Episódios pendurados	fechados com cadeias intactas
Episódios aguardando humano	0
Episódios no tenant do operador	114 (81 encerrados sem ação, 33 selados)
Turnos de recusa, janela de 30 dias	93, dos quais 79 (85%) sob um critério
Sinais cuja evidência nunca ofereceu alvo acionável	10
Contenções com desfecho revisado	4 (1 falso positivo, 3 ausentes)
Passada de revisão pós-correção	por principal; perímetro de hostname relido pelo adaptador dele; veredito CEDO_DEMAIS, motivo registrado

Os números do corpus vêm da própria camada de achados do sistema (§10) e da listagem de sessões dele, e não de um analista consultando o banco — que é o uso pretendido daquela camada e é, em si, um resultado: a medição que mais mudou a nossa conta sobre o corpus (relatada lá) foi produzida pelo componente construído para medir sem poder alterar nada.

A duração é medida entre os carimbos de criação do primeiro e do último turno, que são escritos independentemente. Ela **não** é medida entre os campos `aberto_em` e `selado_em` do episódio: esses dois são escritos a partir de uma única leitura de relógio e são idênticos ao microssegundo, de modo que a diferença entre eles não mede nada. Registramos isto porque um par de campos que parece um intervalo e não é um é o tipo de coisa que vira número de latência publicado em silêncio — e porque a mesma forma aparece na §8 quatro vezes sob nomes diferentes.

Disciplina de construção: doze portões precedem a construção da imagem — testes, lint, independência, limite defensivo, integridade de envelope, terminologia, página do contrato, divergência do SDK de conectores, saída do console, presença de credencial, fronteira de camada, e contrato de API contra código — e dois implantam. A suíte tem 622 testes em 50 arquivos, cada arquivo executado no próprio processo, porque vários substituem funções de módulo para simular falha e uma substituição mal desfeita num processo compartilhado faz o resultado depender da ordem dos arquivos.

O portão de integridade de envelope é aquele cuja falha não é regressão de qualidade. Ele constrói violações de cada predicado do envelope e exige que todas sejam recusadas; uma única admissão significa que aquele predicado não está sendo aplicado, e a resposta certa a um resultado vermelho ali é

parar, não triar.

Dois portões codificam uma lição em vez de um estilo. O portão do limite defensivo roda **antes** dos testes: se a garantia de que o sistema não ataca foi afrouxada, nada do que a suíte relatar importa. O portão de contrato parseia as rotas da árvore sintática da aplicação e as compara com a descrição publicada da API nos dois sentidos, porque um contrato que deriva do código é um contrato que mente para quem integra com ele.

10. O que não está construído

Um inventário de capacidade escrito por quem a construiu não vale nada sem o complemento.

- **Arnês adversarial com holdout cego.** Projetado, não construído. Enquanto não existir, nenhuma afirmação sobre qualidade de política se sustenta, e a §7.4 permanece.
- **Reconciliação contra um mundo de granularidade alheia.** O caso do hostname está resolvido (§6.2); o problema geral não.
- **Eventos de borda por endereço.** Indisponíveis no plano atual da plataforma de aplicação, e a restrição que aperta a diversidade do corpus.
- **Infraestrutura versionada.** Várias definições de tarefa agendada e um papel de execução existem na conta sem declaração versionada; o script de provisionamento ficou atrás da árvore.
- **Um terceiro perímetro.** Esgotado por orçamento de regras, não por desenho (§5.6).

Este inventário parece rápido. Entre a primeira redação destas notas e a primeira revisão delas, no mesmo dia, o sistema executou a primeira contenção real e uma frase afirmando que nenhuma havia ocorrido deixou de ser verdadeira.

11. Conclusão

A DAI age sobre infraestrutura de produção sem humano aprovando cada ação, e o argumento de que isso é admissível não é que o modelo dela é bom. É que a decisão é tomada fora do modelo, por uma função pura de um contrato declarado; que a credencial que verifica é demonstravelmente incapaz de executar; que o perímetro em que ela pode escrever está vinculado ao principal por quem ela age, sem padrão; que a contenção por principal é expressa conjugando um discriminador com a origem, de modo que o raio de alcance é unitário por construção; que todo efeito é alcançável a partir de um registro encadeado por hash que um terceiro consegue recomputar sem o código do sistema; e que todo efeito é revisitado, reconciliado e reversível.

Os defeitos da §8 são a melhor evidência disponível a favor dessa estrutura, porque todos eram silenciosos e todos eram silenciosos pelo mesmo motivo: a garantia morava em convenção e não em construção. A pretensão do sistema à atenção alheia não é que ele não falhe. É que ele falha alto — fail-stop em vez de fail-silent [26, 9].

Limitações

As medições de produção vêm de uma única implantação, ao longo de alguns dias; são instâncias, não estatísticas. As afirmações das §3.3 e §5.4 são arquiteturais e valem para o sistema como construído. A recomputação externa da §4.4 foi feita para um recibo e uma alteração controlada; ela demonstra a propriedade, não a frequência dela. Nada se afirma sobre qualidade de política, que está sem medição (§7.4), e nada sobre suficiência para segurança — apenas que cada componente nomeado é necessário para a garantia que nomeia.

Referências

As entradas são mantidas na língua original das publicações.

- [1] Alahmadi, B. A., Axon, L., Martinovic, I. (2022). *99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms*. Proceedings of the 31st USENIX Security Symposium.
- [2] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., Wilkes, J. (2016). *Borg, Omega, and Kubernetes*. Communications of the ACM 59(5), 50–57.
- [3] Cichonski, P., Millar, T., Grance, T., Scarfone, K. (2012). *Computer Security Incident Handling Guide*. NIST Special Publication 800-61 Revision 2.
- [4] Clark, D. D., Wilson, D. R. (1987). *A Comparison of Commercial and Military Computer Security Policies*. IEEE Symposium on Security and Privacy, 184–194.
- [5] Crosby, S. A., Wallach, D. S. (2009). *Efficient Data Structures for Tamper-Evident Logging*. Proceedings of the 18th USENIX Security Symposium.
- [6] Cvach, M. (2012). *Monitor Alarm Fatigue: An Integrative Review*. Biomedical Instrumentation & Technology 46(4), 268–277.
- [7] Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., Tramèr, F. (2024). *AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents*. arXiv:2406.13352.
- [8] Dennis, J. B., Van Horn, E. C. (1966). *Programming Semantics for Multiprogrammed Computations*. Communications of the ACM 9(3), 143–155.
- [9] Gray, J. (1986). *Why Do Computers Stop and What Can Be Done About It?* Symposium on Reliability in Distributed Software and Database Systems, 3–12.
- [10] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M. (2023). *Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. 16th ACM Workshop on Artificial Intelligence and Security (AISec), 79–90.
- [11] Haber, S., Stornetta, W. S. (1991). *How to Time-Stamp a Digital Document*. Journal of Cryptology 3(2), 99–111.
- [12] Hanappi, O., Hummer, W., Dustdar, S. (2016). *Asserting Reliable Convergence for Configuration Management Scripts*. OOPSLA, 328–343.
- [13] IBM (2006). *An Architectural Blueprint for Autonomic Computing*, 4ª edição. IBM White Paper.

- [14] Islam, C., Babar, M. A., Nepal, S. (2019). *A Multi-Vocal Review of Security Orchestration*. ACM Computing Surveys 52(2), Article 37.
- [15] Kephart, J. O., Chess, D. M. (2003). *The Vision of Autonomic Computing*. IEEE Computer 36(1), 41–50.
- [16] Krebs, R., Momm, C., Kounev, S. (2012). *Architectural Concerns in Multi-Tenant SaaS Applications*. 2nd International Conference on Cloud Computing and Services Science (CLOSER), 426–431.
- [17] Laurie, B., Langley, A., Kasper, E. (2013). *Certificate Transparency*. RFC 6962, IETF.
- [18] Merkle, R. C. (1988). *A Digital Signature Based on a Conventional Encryption Function*. Advances in Cryptology — CRYPTO '87, LNCS 293, 369–378.
- [19] Meszaros, G. (2007). *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley.
- [20] Miller, M. S. (2006). *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. Tese de doutorado, Johns Hopkins University.
- [21] Moreau, L., Groth, P. (2013). *Provenance: An Introduction to PROV*. Morgan & Claypool.
- [22] OASIS (2013). *eXtensible Access Control Markup Language (XACML) Version 3.0*. OASIS Standard.
- [23] Perez, F., Ribeiro, I. (2022). *Ignore Previous Prompt: Attack Techniques for Language Models*. arXiv:2211.09527.
- [24] Saltzer, J. H., Schroeder, M. D. (1975). *The Protection of Information in Computer Systems*. Proceedings of the IEEE 63(9), 1278–1308.
- [25] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., Youman, C. E. (1996). *Role-Based Access Control Models*. IEEE Computer 29(2), 38–47.
- [26] Schlichting, R. D., Schneider, F. B. (1983). *Fail-Stop Processors: An Approach to Designing Fault-Tolerant Computing Systems*. ACM Transactions on Computer Systems 1(3), 222–238.
- [27] Schneier, B., Kelsey, J. (1999). *Secure Audit Logs to Support Computer Forensics*. ACM Transactions on Information and System Security 2(2), 159–176.
- [28] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., Dennison, D. (2015). *Hidden Technical Debt in Machine Learning Systems*. Advances in Neural Information Processing Systems 28.
- [29] Sennrich, R., Haddow, B., Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units*. Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, 1715–1725.
- [30] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems 30.

Autoria: Hiago Kin Levi
Manuscrito original: Brooklyn, NY — 16 de setembro de 2026, 22h00
SHA-256 do corpo do fonte original, até o bloco de autoria:
ae224f4fd28f8d448dfb82c7d1215f628e76084a640ca054993185dcfc229a8e